

FULL STACK DEVELOPMENT

FASE 4 | AULA 02 -

CAMERA

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS	14

EMSE

O QUE VEM POR AÍ?

Nesta aula, vamos explorar como integrar e utilizar a câmera em aplicativos construídos com React Native utilizando Vite. Abordaremos a configuração do ambiente, as bibliotecas necessárias, exemplos de código e as melhores práticas para garantir uma implementação eficiente.

EMAND

HANDS ON

Aprenderemos como a câmera funciona no React Native. Ele utiliza bibliotecas como `react-native-camera` para interagir com APIs nativas do Android e iOS, permitindo o uso da câmera em aplicativos móveis. A biblioteca inicializa a câmera nativa quando montada, solicita permissões do usuário e exibe o feed de vídeo ao vivo na interface. A captura de mídia é feita enviando comandos à câmera nativa, com a mídia retornada ao aplicativo.

No Android, permissões são adicionadas no `AndroidManifest.xml` e no iOS no `Info.plist`. A renderização do feed usa `SurfaceView` ou `TextureView` no Android e `AVCaptureVideoPreviewLayer` no iOS, garantindo uma visualização fluida. Para capturar fotos, usamos o método `takePictureAsync` e para vídeos `recordAsync` e `stopRecording`. A biblioteca também fornece feedback visual e auditivo, melhorando a experiência do usuário.

Configuraremos permissões, inicializaremos a câmera e implementaremos a captura de mídia, garantindo conformidade com as expectativas de privacidade e requisitos da App Store.

SAIBA MAIS

Como a câmera funciona no React Native

Interação com APIs Nativas

O React Native se destaca por permitir a criação de aplicativos móveis utilizando uma única base de código em JavaScript. No entanto, para acessar funcionalidades específicas do dispositivo, como a câmera, ele precisa interagir com APIs nativas do sistema operacional, seja Android ou iOS. Isso é realizado através de bibliotecas como `react-native-camera`, que fornecem uma ponte entre o código JavaScript e as APIs nativas, encapsulando a complexidade dessas interações.

Bibliotecas como `react-native-camera` desempenham um papel crucial ao fornecer uma interface JavaScript simplificada para interagir com a câmera nativa do dispositivo. Essas bibliotecas não apenas facilitam a inicialização e o uso da câmera, mas também cuidam de diversos detalhes técnicos, como a configuração de permissões, a renderização do feed de vídeo e a captura de mídia.

Ciclo de vida da câmera

Quando o componente de câmera é montado no aplicativo, a biblioteca **inicializa** a câmera nativa do dispositivo. Esse processo envolve a configuração de vários parâmetros, como a resolução da câmera, a orientação do dispositivo e outras configurações específicas do hardware.

Para garantir a privacidade e a segurança dos usuários, os aplicativos devem solicitar **permissões** explícitas para acessar a câmera e, em alguns casos, o microfone. No Android, essas permissões são gerenciadas através do arquivo `AndroidManifest.xml`, enquanto no iOS elas são configuradas no `Info.plist`. A biblioteca lida com a lógica necessária para verificar e solicitar essas permissões de forma transparente.

Após obter as permissões necessárias, a biblioteca configura a visualização da câmera no aplicativo. Isso envolve a **exibição do feed** de vídeo ao vivo diretamente na interface do usuário. No Android, são usados elementos como `SurfaceView` ou

TextureView para renderizar o feed da câmera, enquanto no iOS é utilizado o AVCaptureVideoPreviewLayer.

Quando o usuário interage com a interface do aplicativo para tirar uma foto ou gravar um vídeo, a biblioteca envia comandos para a câmera nativa **capturar a mídia**. Esse processo é altamente otimizado para garantir que a mídia capturada seja retornada ao aplicativo com a melhor qualidade possível.

Permissões e segurança

A segurança e a privacidade dos usuários são primordiais ao lidar com funcionalidades sensíveis como a câmera. O React Native exige que as pessoas desenvolvedoras configurem **solicitações de permissões** de maneira explícita. No Android, isso é feito através do AndroidManifest.xml.

A biblioteca inclui métodos para verificar e solicitar permissões dinamicamente, uma **gestão de permissões**. Isso garante que o aplicativo funcione corretamente mesmo que as permissões não tenham sido concedidas previamente. Por exemplo: o método checkPermission verifica se a permissão já foi concedida, enquanto o método requestPermission solicita a permissão ao usuário se necessário.

Renderização do feed da câmera

A renderização do feed da câmera é um aspecto crítico para a experiência do usuário. No Android, SurfaceView e TextureView são comumente usados para essa finalidade. O SurfaceView é um componente de visualização de baixo nível que oferece desempenho eficiente, enquanto o TextureView permite mais flexibilidade na manipulação de gráficos.

No iOS, o AVCaptureVideoPreviewLayer é utilizado para exibir o feed da câmera. Ele é altamente otimizado e integrado com o framework AVFoundation, que oferece uma API robusta para capturar e processar mídia.

Outro ponto importante é a renderização do feed da câmera, que deve ser fluida e responsiva para proporcionar uma boa experiência ao usuário. A biblioteca é projetada para otimizar o desempenho, gerenciando aspectos como taxas de quadros e resolução de forma eficiente. Isso inclui a utilização de threads separadas para

processamento de imagem e vídeo, garantindo que a interface do usuário permaneça responsiva.

Captura de imagens e vídeos

Quando o usuário tira uma foto, a biblioteca envia um comando à câmera nativa para capturar a imagem. A imagem capturada é então processada e retorna ao aplicativo. O processamento pode incluir operações como compressão e ajuste de qualidade para atender aos requisitos específicos do aplicativo.

Já a gravação de vídeos envolve um processo mais complexo em comparação com a captura de fotos. Além de capturar o vídeo, a biblioteca deve configurar codecs de vídeo apropriados e gerenciar o armazenamento dos dados de vídeo. Isso pode incluir a seleção de resoluções de vídeo, taxas de bits e formatos de arquivo adequados para garantir a qualidade e o desempenho.

Feedback e experiência do usuário

Para melhorar a experiência do usuário, a biblioteca consegue fornecer feedback visual e auditivo durante a captura de mídia. Isso pode incluir animações de obturador e sons de clique que simulam o funcionamento de uma câmera tradicional. Esses feedbacks ajudam a confirmar que a ação do usuário foi bem-sucedida e melhoram a usabilidade geral do aplicativo.

Após a captura da mídia, muitos aplicativos oferecem funcionalidades de pré-visualização e edição. Isso permite que usuários revisem e ajustem suas fotos e vídeos antes de salvá-los ou compartilhá-los. A biblioteca react-native-camera facilita a integração dessas funcionalidades, permitindo que os(as) desenvolvedores(as) criem experiências ricas e interativas.

A integração da câmera em aplicativos React Native exige uma compreensão detalhada das interações entre JavaScript e as APIs nativas de Android e iOS. A biblioteca react-native-camera simplifica essa integração, inicializando a câmera e configurando parâmetros específicos do hardware, como SurfaceView e TextureView no Android, e AVCaptureVideoPreviewLayer no iOS, para otimização de renderização e taxa de quadros.

O gerenciamento de permissões é crucial, configurado em AndroidManifest.xml e Info.plist, com a lógica de verificação e solicitação implementada na aplicação. A

captura de mídia é gerenciada eficientemente, com suporte a feedback visual e auditivo, além de funcionalidades de pré-visualização e edição, garantindo uma experiência de usuário rica e responsiva.

Criar um novo projeto React Native com Vite: instale o Vite e crie um novo projeto.

```
npm create vite@latest my-react-native-app --template react
cd my-react-native-app
npm install
```

Configurar o React Native: instale as dependências do React Native.

```
npm install react-native
npm install react-native-cli
```

Instalar a biblioteca react-native-camera.

```
npm install react-native-camera
```

Configurar permissões no Android: abra o arquivo android/app/src/main/AndroidManifest.xml e adicione as permissões a seguir.

```
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.RECORD_AUDIO" />
```

Configurar permissões no iOS: abra o arquivo ios/NomeDoProjeto/Info.plist e adicione o conteúdo a seguir.

```
<key>NSCameraUsageDescription</key>
<string>We need your permission to use the camera</string>
<key>NSMicrophoneUsageDescription</key>
<string>We need your permission to use the microphone</string>
```

Ao incluir <key>NSCameraUsageDescription</key> e <key>NSMicrophoneUsageDescription</key> no arquivo Info.plist do seu projeto iOS,

você está informando ao usuário a necessidade do seu aplicativo de acessar a câmera e o microfone. Essas descrições são essenciais para a transparência e para garantir que o aplicativo atenda às expectativas de privacidade dos usuários e aos requisitos da App Store.

Implementação da câmera

Para a implementação, é necessário configurar a Interface da câmera. Para isso, importe e configure a câmera no componente React Native:

```
import React, { useState, useEffect } from 'react';
import { View, Text, TouchableOpacity } from 'react-native';
import { RNCamera } from 'react-native-camera';

const CameraScreen = () => {
  const [hasPermission, setHasPermission] = useState(null);

  useEffect(() => {
    const requestCameraPermission = async () => {
      const { status } = await Permissions.askAsync(Permissions.CAMERA);
      setHasPermission(status === 'granted');
    };

    requestCameraPermission();
  }, []);

  if (hasPermission === null) {
    return <View />;
  }

  if (hasPermission === false) {
    return <Text>No access to camera</Text>;
  }

  return (
    <View style={{ flex: 1 }}>
      <RNCamera
        style={{ flex: 1 }}
        type={RNCamera.Constants.Type.back}
        flashMode={RNCamera.Constants.FlashMode.on}
        captureAudio={false}
      >
        {{{ camera, status, recordAudioPermissionStatus }} => {
          if (status !== 'READY') return <PendingView />;
          return (
```

```
        <View style={{ flex: 0, flexDirection: 'row', justifyContent: 'center'
    }}>
        <TouchableOpacity onPress={() => takePicture(camera)} style={{ flex:
0, backgroundColor: 'orange', padding: 10 }}>
            <Text style={{ fontSize: 14 }}> SNAP </Text>
        </TouchableOpacity>
    </View>
    );
    }}
    </RNCamera>
</View>
);
};

const takePicture = async (camera) => {
    try {
        const data = await camera.takePictureAsync();
        console.log(data.uri);
    } catch (error) {
        console.error(error);
    }
};

export default CameraScreen;
```

Para capturar fotos, utilize o método `takePictureAsync` da `RNCamera`:

```
const takePicture = async (camera) => {
    try {
        const data = await camera.takePictureAsync();
        console.log(data.uri);
    } catch (error) {
        console.error(error);
    }
};
```

Para capturar vídeos, utilize os métodos `recordAsync` e `stopRecording`:

```
const recordVideo = async (camera) => {
```

```
try {
  const data = await camera.recordAsync();
  console.log(data.uri);
} catch (error) {
  console.error(error);
}
};

const stopRecording = async (camera) => {
  try {
    await camera.stopRecording();
  } catch (error) {
    console.error(error);
  }
};
```

Além disso, também há como adicionar botões para captura de foto e vídeo:

```
return (
  <View style={{ flex: 1 }}>
    <RNCamera
      style={{ flex: 1 }}
      type={RNCamera.Constants.Type.back}
      flashMode={RNCamera.Constants.FlashMode.on}
      captureAudio={true}
    >
      {{{ camera, status, recordAudioPermissionStatus }} => {
        if (status !== 'READY') return <PendingView />;
        return (
          <View style={{ flex: 0, flexDirection: 'row', justifyContent: 'center'
}}}
            <TouchableOpacity onPress={() => takePicture(camera)} style={{ flex: 0,
backgroundColor: 'orange', padding: 10 }}>
              <Text style={{ fontSize: 14 }}> SNAP </Text>
            </TouchableOpacity>
            <TouchableOpacity onPress={() => recordVideo(camera)} style={{ flex: 0,
backgroundColor: 'red', padding: 10 }}>
              <Text style={{ fontSize: 14 }}> RECORD </Text>
            </TouchableOpacity>
            <TouchableOpacity onPress={() => stopRecording(camera)} style={{ flex:
0, backgroundColor: 'gray', padding: 10 }}>
              <Text style={{ fontSize: 14 }}> STOP </Text>
            </TouchableOpacity>
          </View>
        )
      }}
    </RNCamera>
  </View>
);
```

```
        </View>
      );
    }}
    </RNCamera>
  </View>
);
```

EMENDADO

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, abordamos como integrar e utilizar a câmera em aplicativos React Native utilizando Vite. Exploramos a configuração inicial do projeto, a instalação e configuração da biblioteca react-native-camera e a implementação de funcionalidades de captura de foto e vídeo. Também discutimos as melhores práticas para garantir uma boa experiência de usuário.



REFERÊNCIAS

DABIT, N. **React Native in Action**. Shelter Island: Manning Publications, 2019.

EISENMAN, B. **Learning React Native**: Building Native Mobile Apps with JavaScript. Sebastopol: O'Reilly Media, 2017.

LEBENSOLD, J.; MEBANE, D. **React Native Cookbook**: Bringing the Web to Native Platforms. Sebastopol: O'Reilly Media, 2018.

EXEMPLO

PALAVRAS-CHAVE

Palavras-chave: React native. Front-end. Câmera.

EMSE

POS TECH